Deslop: Code Quality Analysis Command

Note: This file is 27k tokens as of 2026-01-19

> A comprehensive slash command for identifying and fixing "slop" in your codebase. If you use Claude Code, drop this file into your `~/.claude/commands/` folder. If you use OpenCode, put it in `~/.opencode/commands/` folder. Then, restart your agent harness and run `/deslop`.

This command combines a code analysis workflow with an extensive library of coding principles. When you run `/deslop [file-or-directory]`, or even just `/deslop` or perhaps `/deslop my frontend typescript code` the AI will read your code, cross-reference it against these principles, and suggest specific fixes with before/after examples.

Just download this entire markdown file into the location for your agent harness's config folder (such as below) and then restart your harness and run `/deslop`. - `~/.claude/commands/deslop.md` - `~/.opencode/commands/deslop.md`

Whether or not you use this deslop command on your code base, you should read all the coding principles yourself, as a human - you might actually learn something useful.

Table of Contents

1. The Deslop Command
2. Coding Principles Reference
    - Part I: Clean Code — *Writing clear, simple, readable code*
        - Simplicity & Minimalism — do less, but better
            - KISS — avoid unnecessary complexity
            - YAGNI — don't build until needed
            - Small Functions — short, focused, one purpose
            - Guard Clauses — exit early for invalid states
        - Clarity & Readability — code is read 10x more than written
            - Cognitive Load — reduce mental effort to understand
            - Single Level of Abstraction — don't mix abstraction levels
            - Self-Documenting Code — names reveal purpose
            - Documentation Discipline — code tells how, comments tell why
            - Elegance — beauty through insight and minimality
            - Least Surprise — behave as users expect
    - Part II: Architecture — *Structuring and designing systems*
        - Organization & Structure — where does this code belong?
            - DRY — one authoritative representation per concept
            - Single Source of Truth — one location for each piece of data

        - Separation of Concerns — one responsibility per component
        - Modularity — independent components with hidden internals
    - Coupling & Dependencies — how components relate to each other
        - Encapsulation — bundle data with behavior, hide internals
        - Law of Demeter — only talk to immediate friends
        - Orthogonality — changes in one don't affect others
        - Dependency Injection — pass dependencies, don't create them
        - Composition Over Inheritance — combine objects, don't extend classes
    - Design Patterns & Conventions — proven approaches to common problems
        - SOLID — five foundational OO design principles
        - Convention Over Configuration — sensible defaults, override when needed
        - Command-Query Separation — return value OR change state, not both
        - Code Reusability — earned through proven need, not designed upfront
    - Data & State — how data flows and behaves
        - Parse, Don't Validate — transform data into types that prove validity
        - Immutability — once created, state cannot change
        - Idempotency — multiple executions produce same result as one
- Part III: Reliability — *Building robust, maintainable systems*
    - Robustness & Safety — handling the unexpected
        - Fail-Fast — detect and report errors immediately
        - Design by Contract — explicit agreements between callers and routines
        - Postel's Law — conservative output, liberal input
        - Resilience — continue operating despite partial failures
        - Least Privilege — minimum permissions necessary
    - Maintainability & Operations — keeping systems healthy over time
        - Boy Scout Rule — leave code better than you found it
        - Observability — understand what systems do in production

3. When to Relax Rules — context over dogma
4. References — foundational texts and resources
    - Foundational Texts — essential books that shaped software design
    - Seminal Articles & Essays — influential online writings
    - Online Resources — living references for patterns and principles
    - Concept Attribution — origins of specific ideas in this document

---

The Deslop Command

↑ top

You are a code quality analyzer. Your task is to identify "slop" - code that violates established coding principles - and suggest concrete improvements.

At the end, figure out what you should actually change in the code and ask the user if you should make the changes. Then make the changes if the user affirms.

Target

Analyze: $ARGUMENTS

If no argument provided, operate on the current folder or current code base.

Process

1. **Read all coding principles** from this document to understand what good code looks like.
2. **Read the target file(s)** using the Read tool
3. **Reread relevant coding principles** based on what violations you observe
4. **Identify violations** organized by principle
5. **Suggest concrete fixes** with before/after examples

Output Format

Summary Brief overview of code health (1-2 sentences).

Violations Found For each violation:

```
##### [Principle Name] - [Specific Issue]

**Location**: `file.py:line_number`

**Problem**: [Description of what's wrong]

**Before**:
```python
# problematic code
```

After:

```
# improved code
```

Why: [Brief explanation referencing the principle]
```

Recommendations Prioritized list of changes, most impactful first.

Then, ask the user if they'd like to implement some or all of the changes.

If they affirm, then implement them next. When implementing them, consider if some of the changes could be implemented in parallel with async agents for efficiency.

Important Notes

- **Don't over-engineer**: Suggesting abstractions for single-use code violates YAGNI/KISS
- **Context matters**: Test code has different standards (DAMP over DRY)
- **Rule of Three**: Don't suggest abstracting until pattern proven with 3+ occurrences
- **Incidental similarity is not duplication**: Don't merge code that happens to look similar but represents different concepts
- **Be specific**: Reference exact line numbers and provide concrete before/after code

Priority Matrix

Prioritize fixes by impact and effort.

Priority	Type	Examples	Fix When
P0: Critical	Security, data loss	SQL injection, unvalidated input, race conditions	Immediately
P1: High	Bugs waiting to happen	Missing error handling, silent failures, unclear ownership	This PR
P2: Medium	Maintainability	DRY violations (3+), god classes, deep nesting	When touching file
P3: Low	Polish	Magic numbers, naming, minor duplication	If time permits
P4: Optional	Style	Formatting, comment cleanup, minor refactors	Boy Scout Rule

Effort modifiers: - **Quick win** ($<$ 5 min): Bump up one priority level - **Risky change** (no tests): Bump down one level, suggest adding tests first - **Requires coordination**: Note in recommendations, may need team discussion

Example Output

Summary The module has good structure but contains several DRY violations and magic numbers that reduce maintainability.

Violations Found

Self-Documenting Code - Magic Numbers **Location**: `processor.py:45-48`

Problem: Hardcoded numeric values without explanation

Before:

```
if retry_count > 3:
    time.sleep(0.5)
```

After:

```
MAX_RETRIES = 3
RETRY_DELAY_SECONDS = 0.5

if retry_count > MAX_RETRIES:
    time.sleep(RETRY_DELAY_SECONDS)
```

Why: Named constants are self-documenting and centralize configuration.

DRY - Duplicated Validation Logic **Location**: `api.py:23-28` and `api.py:67-72`

Problem: Same email validation logic in two places

Before:

```
# In create_user():
if not email or '@' not in email:
    raise ValueError("Invalid email")

# In update_user():
if not email or '@' not in email:
    raise ValueError("Invalid email")
```

After:

```
def validate_email(email: str) -> None:
    if not email or '@' not in email:
        raise ValueError("Invalid email")

# In both functions:
validate_email(email)
```

Why: Same business rule duplicated - if validation changes, both must update.

Recommendations

1. Extract `validate_email()` helper (DRY - affects 2 locations)
2. Replace magic numbers with named constants (Self-Documenting - affects 4 locations)
3. Consider splitting `UserManager` into `UserService` and `UserRepository` (SRP - optional, low priority)

Coding Principles Reference

↑ top

Quick Diagnostic Guide

See a symptom? Jump to the relevant principle.

Symptom	Likely Principle	Quick Fix
Function > 50 lines	Small Functions	Extract named helpers
Deep nesting (3+ levels)	Guard Clauses, Cognitive Load	Early returns
Copy-pasted code (3+ times)	DRY	Extract shared function
Magic numbers/strings	Self-Documenting Code	Named constants
Class doing many things	SOLID (SRP), Separation of Concerns	Split by responsibility
Long parameter lists (5+)	Encapsulation	Parameter object
`a.b().c().d()` chains	Law of Demeter	Delegate to intermediate
Speculative features	YAGNI	Delete until needed
Comments explaining "what"	Self-Documenting Code	Rename to be obvious
Stale/wrong comments	Documentation Discipline	Delete or fix
Same data in multiple tables	Single Source of Truth	Designate authoritative source
Tests require complex setup	Dependency Injection	Inject dependencies
Inheritance hierarchy > 2 deep	Composition Over Inheritance	Compose objects

Symptom	Likely Principle	Quick Fix
Boolean parameters	Small Functions, KISS	Separate functions
Inconsistent error handling	Fail-Fast	Validate at entry
Silent failures	Fail-Fast, Observability	Fail loudly, log
Getters exposing internals	Encapsulation	Tell, don't ask

Principle Tensions

Principles sometimes conflict. Here's how to resolve common tensions.

Tension	Resolution
DRY vs. Coupling	Duplication is cheaper than wrong abstraction. Wait for Rule of Three. If abstracting requires parameters/conditionals to handle differences, keep separate.
YAGNI vs. Extensibility	Build for today, but keep code malleable. Don't add extension points; ensure code is easy to modify when needed.
KISS vs. DRY	Three lines of obvious code beats one line of clever abstraction. Optimize for reader comprehension.
Fail-Fast vs. Resilience	Fail fast for bugs (programmer errors). Retry/degrade for operational failures (network, disk).
Encapsulation vs. Testing	Prefer testing through public interface. If you need to test internals, the design may need work.
Postel's Law vs. Fail-Fast	Be liberal on input *format* (accept trailing whitespace), but strict on *required data* (reject missing fields).
Small Functions vs. Cognitive Load	Too many tiny functions forces readers to jump around. Balance: functions should do one thing, but that "thing" can be substantial.
DRY vs. Decoupling	Shared code creates coupling. If two teams/services share code, changes affect both. Sometimes copy-paste is correct for independence.

Tension	Resolution
Convention vs. Explicitness	Conventions reduce boilerplate but hide behavior. Document conventions; allow overrides.

Anti-Pattern Quick Reference

Fast detection of common code smells.

Anti-Pattern	Symptoms	Violates
God Class	500+ lines, "Manager"/"Handler" suffix, does everything	SRP, Modularity
Feature Envy	Method uses another class's data more than its own	Encapsulation
Shotgun Surgery	One change requires edits in 10+ files	Separation of Concerns
Primitive Obsession	Passing `(str, str, int)` instead of `User` object	Encapsulation
Data Clumps	Same 3-4 params always passed together	Encapsulation
Long Method	Function > 50 lines, multiple levels of abstraction	Small Functions
Speculative Generality	Unused interfaces, "for future use" code	YAGNI
Dead Code	Unreachable code, unused functions	YAGNI, Boy Scout
Magic Numbers	`if x > 86400` instead of `SECONDS_PER_DAY`	Self-Documenting
Inappropriate Intimacy	Class accesses another's private details	Encapsulation, LoD
Message Chains	`a.getB().getC().getD()`	Law of Demeter
Middle Man	Class delegates everything, adds no value	KISS
Refused Bequest	Subclass doesn't use inherited methods	Liskov, Composition

Anti-Pattern	Symptoms	Violates
Comments as Deodorant	Comments explaining bad code instead of fixing it	Self-Documenting
Cargo Cult	Patterns used without understanding why	KISS, YAGNI

Part I: Clean Code

↑ top

> *Writing clear, simple, readable code. These principles govern the act of writing code itself—making each line, function, and file as clear as possible.*

Simplicity & Minimalism

↑ top

The foundation of clean code: do less, but better. These principles share a core insight—complexity is the enemy. KISS provides the philosophy, YAGNI prevents premature complexity, Small Functions break down complexity, and Guard Clauses eliminate nested complexity.

KISS: Keep It Simple, Stupid

↑ top

> "Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away." — Antoine de Saint-Exupéry

Core Concept

KISS is the discipline of **avoiding unnecessary complexity**. Coined by Kelly Johnson at Lockheed Skunk Works (1960), the principle states systems work best when kept simple.

Two sins of complexity: 1. **Too many parts** in the system 2. **Too many interconnected parts** coupling the system together

Simple Easy: Simple systems have few interconnected parts. Easy tasks require little effort.

Measuring Complexity

Metric	Measures	Threshold	Use Case
Cyclomatic Complexity	Independent paths through code	10/function	Test planning
Cognitive Complexity	Mental effort to understand	15/function	Readability

Common Violations

Code Smells: Single-implementation interfaces, factories of factories, deep inheritance, "clever" one-liners.

Verbal Cues: "This pattern will be useful when…", "Let me make this more flexible…", "This is the proper enterprise way…"

Four Classes of Violations

Class	Example
Cleverness Over Clarity	Nested ternaries, regex golf
Premature Optimization	Caching before profiling
Unnecessary Abstraction	Interface for single implementation
Speculative Generality	Calculator with plugin architecture

Anti-Patterns

```python
#  Wrong - Over-engineered calculator
class OperationInterface(ABC):
    @abstractmethod
    def execute(self, a: float, b: float) -> float: ...

class AddOperation(OperationInterface):
    def execute(self, a, b): return a + b

class OperationFactory:
    def create(self, op: str) -> OperationInterface: ...

#  Correct - Direct solution
def calculate(a: float, b: float, op: str) -> float:
    if op == '+': return a + b
```

```
    if op == '-': return a - b
    if op == '*': return a * b
    if op == '/': return a / b
    raise ValueError(f"Unknown operation: {op}")
```

The Simplicity Test

Before adding complexity: **Can a junior understand this?** — **Does it solve a problem we have today?** — **Am I trying to impress or communicate?**

Summary

1. **Fewer parts, fewer connections** — complexity kills maintainability
2. **Simple Easy** — simple systems may require skill to build
3. **Junior-readable code** — if they can't understand it, it's too complex
4. **Hardcode first** — add configurability when proven necessary (see also: YAGNI)
5. **Measure complexity** — cyclomatic 10, cognitive 15 per function
6. **Simplest sufficient code** — not incomplete, not over-engineered

---

YAGNI: You Aren't Gonna Need It

↑ top

> "Always implement things when you actually need them, never when you just foresee that you need them." — Ron Jeffries, XP co-founder

Core Concept

YAGNI is the discipline of **not building functionality until it's required**. Every feature has costs: development, testing, maintenance, cognitive load. Features you don't need yet carry these costs without delivering value.

The trap: "While I'm here, I'll just add…" — **The reality**: of speculative features fail to improve their target metrics.

Four Costs of YAGNI Violations

Cost	Description
Build	Time developing, testing, debugging unused code
Delay	Value lost by not building needed features instead
Carry	Complexity slowing all future development
Repair	Fixing when requirements differ from predictions

When YAGNI Applies

Apply YAGNI	Don't Apply YAGNI
Speculative features	Security (build in from start)
"Just in case" abstractions	Logging/observability
Unused configuration options	API versioning (public APIs)
Premature optimization	CI/CD and testing infrastructure
Generic frameworks for single use	Data migration paths

Common Violations

Code Smells: Config options no one uses, ABC with one implementation, extensibility points never extended, commented "future" code, unused API endpoints.

Verbal Cues: "We might need this later", "Just in case", "While we have the hood open…", "For future flexibility…"

Anti-Patterns

```python
#  Wrong - Speculative abstraction
class DataExporter(ABC):
    @abstractmethod
    def export(self, data): ...

class JSONExporter(DataExporter):
    def export(self, data): return json.dumps(data)
# CSVExporter, XMLExporter never built...

#  Correct - Build what you need
def export_to_json(data):
    return json.dumps(data)
# Add abstraction when second exporter is needed
```

The Delete Test

Before adding code: **Who needs this today?** (not "might need") — **What breaks without it?** (if nothing, skip it) — **Can we add it later?** (usually yes, with better understanding)

Summary

1. **Build only what's needed now** — of speculative features fail
2. **Delete speculative code** — git has history
3. **Hardcode first** — configure when needed (see also: KISS)

4. **Concrete over abstract** — until third occurrence (see also: DRY's Rule of Three)
5. **Keep code malleable** — YAGNI requires easy-to-change code

---

Small Functions

↑ top

> "The first rule of functions is that they should be small. The second rule of functions is that they should be smaller than that." — Robert C. Martin (Uncle Bob), *Clean Code*

Core Concept

Small Functions is the principle that **functions should be short, focused, and do one thing well**. Decompose logic into small, named units that can be understood at a glance.

The key insight: If you spend effort figuring out what code does, extract it into a function and name it after that "what." The name becomes documentation.

Guideline sizes (not rigid rules): - **Ideal**: 5-15 lines - **Warning**: 20-30 lines - **Smell**: 50+ lines

Why Small Functions Work

Large Functions	Small Functions
Hard to name (does too many things)	Easy to name (does one thing)
Multiple levels of abstraction	Single level of abstraction
Difficult to test in isolation	Easy to unit test
Changes risk breaking unrelated logic	Changes are localized

The Stepdown Rule

Code should read like a top-down narrative, descending one level of abstraction at a time:

```
#  Correct - Reads like a story
def process_order(order: Order) -> Receipt:
    validate_order(order)
    apply_discounts(order)
    charge_payment(order)
    send_confirmation(order)
    return create_receipt(order)
```

Common Violations

Code Smells: - Functions over 30 lines - Multiple `# Section` comments within one function - Deeply nested conditionals (3+ levels) - Functions with "And" in the name (`validateAndSave`)

Verbal Cues: - "This function is long but it's all related" - "Let me add a comment to explain this section" - "I'll refactor it later when we have time"

When NOT to Apply

The Counterargument (Cindy Sridharan's "Small Functions Considered Harmful"): - **Loss of locality**: Jumping across many files increases cognitive load - **Naming explosion**: More functions = more names to invent and remember - **Shallow modules**: Many trivial functions can be worse than fewer deep ones

When larger functions are acceptable: - Sequential logic that must share context - State machines hard to decompose without passing lots of state - Performance-critical code where call overhead matters - One-off scripts that won't be maintained

The test: Can a newcomer understand this function in one read? If yes, it's fine—regardless of line count.

Anti-Patterns

```python
#  Too shallow - interface complexity exceeds implementation
def is_empty(collection): return len(collection) == 0
def is_not_empty(collection): return len(collection) > 0

#  Better - meaningful abstraction hiding complexity
def get_active_users(user_ids: list[int]) -> list[User]:
    """Fetches users, filters inactive, sorts by last_active."""
    users = fetch_users_batch(user_ids)
    active = [u for u in users if u.is_active]
    return sorted(active, key=lambda u: u.last_active, reverse=True)
```

Relationship to Other Principles

Principle	Connection
Single Responsibility	Small Functions is the *how*, SRP is the *what*
Separation of Concerns	Decompose by concern, then make each piece small
DRY	Extract duplicated code into small reusable functions

Principle	Connection
Self-Documenting Code	Function names replace comments when functions are small
KISS	Small functions are simpler to understand

Summary

1. **Keep functions short** — 5-20 lines is a good target, 50+ is a smell
2. **One level of abstraction** — Don't mix high-level flow with low-level details
3. **Name the "what"** — Extract code and name the function after its purpose
4. **Balance depth vs. breadth** — Avoid shallow modules with trivial functions
5. **Optimize for the reader** — Newcomers should understand the code quickly

---

Guard Clauses (Early Return)

↑ top

> "The guard clause says, 'This is rare, and if it happens, do something and get out.' " — Martin Fowler, *Refactoring*

Core Concept

Early exit when preconditions aren't met. Check invalid states at the top, return immediately. Keeps the "happy path" at outermost indentation.

Fights rightward drift—the "arrow anti-pattern":

```
if () {
    if () {
        do {
            if () {
                if () {
                    // actual logic buried here
                }
            }
        }
    }
}
```

Guard clauses flatten this by handling exceptions first.

The Transformation

```
#  Wrong - Nested conditionals obscure the happy path
def get_pay_amount(employee):
    result = 0
    if employee.is_dead:
        result = dead_amount()
    else:
        if employee.is_separated:
            result = separated_amount()
        else:
            if employee.is_retired:
                result = retired_amount()
            else:
                result = normal_pay_amount()
    return result

#  Correct - Guard clauses make special cases obvious
def get_pay_amount(employee):
    if employee.is_dead:
        return dead_amount()
    if employee.is_separated:
        return separated_amount()
    if employee.is_retired:
        return retired_amount()
    return normal_pay_amount()
```

When to Use

1. **Precondition validation** — null checks, empty inputs, invalid states
2. **Edge case handling** — special states that bypass normal logic
3. **Base cases** — recursive function termination

```
#  Classic guard clause pattern
def send_welcome_email(user):
    if user is None:
        return
    if not user.email:
        return

    # Main logic at natural indentation
    mailer.send(user.email, "Welcome!")
```

When NOT to Use

When both branches are equally valid, use conventional conditionals:

```python
#  Misleading - Both branches are equally valid
def process_order(order):
    if order.is_express:
        return handle_express_shipping(order)
    return handle_standard_shipping(order)

#  Better - if/else signals equal weight
def process_order(order):
    if order.is_express:
        handle_express_shipping(order)
    else:
        handle_standard_shipping(order)
```

A guard clause signals "this is unusual—handle it and leave." Equal-weight branches deserve equal-weight syntax.

The Single-Return Myth

Some codebases enforce "single return point" rules—a practice from Dijkstra's era when early returns could cause resource leaks in C. In modern languages with garbage collection and `try/finally`, this constraint is obsolete. The single-return style forces mutable state to accumulate results:

```python
#  Single-return requires mutable state
def validate(data):
    result = True
    if not data.get('name'):
        result = False
    if result and not data.get('email'):
        result = False
    return result

#  Guard clauses are cleaner
def validate(data):
    if not data.get('name'):
        return False
    if not data.get('email'):
        return False
    return True
```

Common Violations

Guard clause buried in the middle:

```python
#  Wrong - Guards belong at the top
def process(item):
    item.prepare()
    item.validate()
```

```
    if not item.is_ready:  # Too late
        return None
    return item.execute()

#  Correct - Check preconditions first
def process(item):
    if not item.can_process:
        return None
    item.prepare()
    item.validate()
    return item.execute()
```

Relationship to Other Principles

Principle	Relationship
Fail-Fast	Guard clauses are fail-fast's implementation: detect problems immediately and exit
Cognitive Load	Flattening nested conditionals reduces mental overhead
Small Functions	Guards work best in small, focused functions
Design by Contract	Guards enforce preconditions at runtime

Summary

1. **Exit early for exceptional cases** — handle invalid states at the top
2. **Flatten nested conditionals** — each guard removes a nesting level (see also: Cognitive Load)
3. **Signal intent** — guards = "unusual," if/else = "both paths normal"
4. **Keep guards at the entrance** — preconditions belong at the top
5. **Embrace multiple returns** — single-return is obsolete

Clarity & Readability

↑ top

Code is read 10x more than it's written. Cognitive Load explains why clarity matters (working memory limits), while Self-Documenting Code and Documentation Discipline provide the how. Elegance adds the aesthetic dimension, and Principle of Least Surprise ensures predictability.

Cognitive Load

↑ top

> "Cognitive load is how much a developer needs to think in order to complete a task." — Artem Zakirullin

Core Concept

Mental effort to understand code. Working memory holds ~**4 chunks**; exceed this and comprehension fails. **We read code 10x more than we write it**—every clever trick forces readers to hold more in their head.

Three Types of Load

Type	Description	Reducible?
Intrinsic	Inherent task difficulty	No
Extraneous	How info is presented	**Yes—focus here**
Germane	Builds understanding	Desirable

Common Violations

```python
#  Wrong - Each condition fills working memory
if val > THRESHOLD and (cond_a or cond_b) and (cond_c and not cond_d):
    process(val)  #  Reader is lost

#  Correct - Named intermediates free working memory
is_above_threshold = val > THRESHOLD
is_allowed = cond_a or cond_b
is_secure = cond_c and not cond_d

if is_above_threshold and is_allowed and is_secure:  #  Fresh
    process(val)
```

```python
#  Wrong - Deep nesting accumulates load
if is_valid:           #  +
    if is_authorized:  #  ++
        if has_quota:  #  +++
            process()  #

#  Correct - Early returns keep memory clear
if not is_valid:
    return
if not is_authorized:
    return
if not has_quota:
```

```
    return
process()  #  All preconditions met
```

The Familiarity Trap

Familiarity simplicity. Code in long-term memory feels easy; newcomers face full burden.

Symptom	Reality
"It makes sense once you understand our patterns"	High learning curve = high load
"It's not that complicated"	Your long-term memory is doing the lifting

Deep vs. Shallow Modules

Type	Interface	Implementation	Cognitive Load
Deep	Simple	Complex	Low—complexity hidden
Shallow	Complex	Simple	High—overhead exceeds value

Unix I/O: five functions (`open`, `read`, `write`, `lseek`, `close`) hiding hundreds of thousands of lines. Contrast with `MetricsProviderFactoryFactory`—the name alone is more taxing than the implementation.

Anti-Patterns

Anti-Pattern	Problem
Too many tiny files	Must hold all 80 class interactions in mind
Layered architecture for its own sake	Each indirection layer adds overhead
Clever one-liners	Reader must recreate author's thought process
Premature microservices	Distributed debugging is exponentially harder

Relationship to Other Principles

Principle	Connection
KISS	Cognitive load is *why* simplicity matters
Self-Documenting Code	Good names reduce mental translation
Small Functions	Must balance: too many shallow functions *increase* load
Composition Over Inheritance	Explicit dependencies reduce hidden context
Modularity	Deep modules hide complexity behind simple interfaces

Summary

1. **Working memory holds ~4 chunks** — Exceed this and comprehension fails
2. **Reduce extraneous load** — Focus on how code is presented
3. **Familiarity simplicity** — Code you know feels easy; newcomers feel the burden
4. **Prefer deep modules** — Simple interfaces hiding complex implementations
5. **Write boring code** — The best code requires no mental effort to parse

---

Single Level of Abstraction (SLAP)

↑ top

> "The code within a function should operate at a single level of abstraction." — Robert C. Martin, Clean Code

Core Concept

Single Level of Abstraction Principle (SLAP) states that **every statement within a function should operate at the same level of abstraction**. When you mix high-level operations (like "process order") with low-level details (like "parse JSON field"), the code becomes harder to read because readers must mentally switch between abstraction levels.

The key insight: Switching between levels of abstraction forces mental grouping—readers must mentally construct the missing abstractions by finding which statements belong together.

Abstraction Levels

Level	Examples
High	`process_order()`, `authenticate_user()`, `generate_report()`
Medium	`validate_email()`, `calculate_tax()`, `format_response()`
Low	`strip().upper()`, `int(value)`, `encode('utf-8')`

Common Violations

```python
#  Wrong - Mixed abstraction levels
def process_order(order_data: dict) -> None:
    user = get_user(order_data["user_id"])  # High-level

    # Low-level detail mixed in
    items = []
    for item in order_data.get("items", []):
        items.append({
            "sku": item["sku"].strip().upper(),
            "qty": int(item.get("quantity", 1))
        })

    validate_inventory(items)  # High-level
    charge_payment(user, calculate_total(items))
    send_confirmation(user)

#  Correct - Single level of abstraction
def process_order(order_data: dict) -> None:
    user = get_user(order_data["user_id"])
    items = parse_order_items(order_data)
    validate_inventory(items)
    charge_payment(user, calculate_total(items))
    send_confirmation(user)
```

The Stepdown Rule

Robert Martin's Stepdown Rule: code should read like a top-down narrative. Each function leads to the next level of abstraction, like a newspaper article—headline first, then summary, then details.

```python
#  Reads top-down at consistent level
def generate_monthly_report(month: int, year: int) -> Report:
    data = fetch_monthly_data(month, year)
    metrics = calculate_metrics(data)
    charts = generate_visualizations(metrics)
    return compile_report(metrics, charts)
```

Detecting Violations

Smell #1: Loops with inline logic

```
#  Extract the loop body
for entity in entities:
    dto = ResultDto()
    dto.shoe_size = entity.shoe_size
    dto.age = compute_age(entity.birthday)
    results.append(dto)

#  Single statement in loop
for entity in entities:
    results.append(to_dto(entity))
```

Smell #2: Comment + code block

```
#  Comment indicates missing abstraction
# Validate email format
if not re.match(r'^[\w.-]+@[\w.-]+\.\w+$', email):
    raise ValueError("Invalid email")

#  Named function replaces comment
validate_email_format(email)
```

Caveats

- **Mental inlining**: Over-extraction forces readers to jump between many tiny functions
- **Simple code doesn't need extraction**: A 3-line function is already at one level
- **Guard clauses are OK**: An initial `if param is None: raise` at a higher-level function is acceptable
- **Performance**: Sometimes inlining is necessary for hot paths

When NOT to Apply

- **Test code**: Explicit inline steps improve test readability
- **Single-use transformations**: Don't extract if it obscures more than clarifies
- **Trivially simple functions**: Extraction for its own sake adds noise

Summary

1. **Every statement at the same abstraction level** — Don't mix orchestration with implementation
2. **Extract when you see mixing** — Loops with logic, comments + code blocks

3. **Use the stepdown rule** — High-level functions call medium-level, which call low-level
4. **Avoid over-extraction** — Balance SLAP against readability (see also: Small Functions, Cognitive Load, Separation of Concerns)

Self-Documenting Code

↑ top

> "Any fool can write code that a computer can understand. Good programmers write code that humans can understand." — Martin Fowler

Core Concept

Code that **conveys purpose** through names, structure, and organization—without relying on comments. Comments explain *why*, code shows *what*.

Reveals: What/how (through naming and structure) · **Cannot reveal:** Why/context (requires comments/docs)

The Three Pillars

1. Intention-Revealing Names Names express purpose, not implementation. **Spell words out completely**—abbreviations force mental translation.

```
#  Wrong
def proc(d, w):
    return d * w * 8

#  Correct
def calculate_billable_hours(days_worked: int, weeks: int) -> int:
    hours_per_day = 8
    return days_worked * weeks * hours_per_day
```

2. Eliminate Magic Values Replace hardcoded numbers with named constants.

```
#  Wrong
if retry_count > 3:
    time.sleep(0.5)
```

```
#  Correct
MAX_RETRIES = 3
RETRY_DELAY_SECONDS = 0.5
if retry_count > MAX_RETRIES:
    time.sleep(RETRY_DELAY_SECONDS)
```

3. Structured Organization Each function has one clear purpose. Structure tells the story.

Naming Conventions

Element	Convention	Examples
Variables	Nouns, fully spelled out	`user_count`, `retry_delay_seconds`
Functions	Verbs/verb phrases	`calculate_total()`, `validate_input()`
Predicates	`is_`, `has_`, `can_` prefix	`is_active`, `has_permission`
Classes	Nouns, PascalCase	`UserAccount`, `OrderProcessor`
Constants	UPPER_SNAKE_CASE	`MAX_RETRIES`, `DEFAULT_TIMEOUT`

Common Violations

Code Smells: Abbreviations (`usr`, `cnt`), single-letter variables outside tiny scopes, boolean parameters without names, vague function names (`process`, `handle`, `do`).

The Comment Balance

Self-documenting handles **what/how**. Comments handle **why/why not**.

```
#  Correct - Comment explains why
# Exponential backoff: upstream API rate-limits during peak hours (ISSUE-1234)
for attempt in range(MAX_RETRIES):
    time.sleep(2 ** attempt)
```

Summary

1. **Spell out names completely** — `user_count` not `usr_cnt` (reduces Cognitive Load)
2. **Eliminate magic values** — named constants explain meaning
3. **Structure tells story** — one function, one purpose
4. **Code shows what/how** — comments explain why/why not (see also: Documentation Discipline)

Documentation Discipline

↑ top

> "Code tells you how, comments tell you why." — Jeff Atwood, Stack Overflow co-founder

Core Concept

Right documentation at the right level. Comments don't compile, can't be tested, and rot—yet sometimes they're essential for explaining "why." The discipline: knowing the difference.

The Documentation Pyramid

Layer	Audience	Purpose
README	New users/devs	First contact, setup, overview
API Docs	Consumers	Contract, usage, edge cases
Docstrings	Callers	What it does, params, returns
Inline Comments	Maintainers	Why this specific implementation

Move documentation to the highest appropriate level.

When Comments Add Value

```
#  Why - Business logic rationale
# Orders over $1000 require manager approval per SOX compliance (POLICY-2019-04)
if order.total > MANAGER_APPROVAL_THRESHOLD:
    require_approval(order)

#  Why not - Explaining rejected alternatives
# Using linear search instead of binary: list is always <10 items
# and maintaining sort order would cost more than the lookup savings

#  Workarounds - External constraints
# Firefox doesn't fire mouse events when dragging outside the window.
# Workaround: capture position on mouseLeave and extrapolate.

#  Links - Attribution and context
# Algorithm from https://stackoverflow.com/a/46018816 (CC-BY-SA)

#  Warnings - Prevent future mistakes
# Don't use global isFinite()-it returns true for null values
Number.isFinite(value)
```

Comment Anti-Patterns

Anti-Pattern	Problem	Fix
Parrot comments	`i += 1  # increment i`	Delete—code already says this

Anti-Pattern	Problem	Fix
Rotting comments	Comment describes deleted code	Delete or update
Journal comments	`// Fixed by John, 3/15`	Use git blame instead
Commented-out code	Dead code polluting the file	Delete—git has history
Closing brace comments	`} // end if`	Extract to smaller functions
Mandated comments	Boilerplate on every method	Comment only when valuable
TODO graveyards	`// TODO: fix this (2019)`	Create tickets or delete

```python
#  Wrong - Parrot comment
def calculate_tax(amount):
    tax_rate = 0.08  # Set tax rate to 0.08
    return amount * tax_rate  # Return amount times tax rate

#  Correct - Explains the why
def calculate_tax(amount):
    # California state tax rate as of 2024. Updates tracked in POLICY-TAX-01.
    CA_TAX_RATE = 0.08
    return amount * CA_TAX_RATE
```

The Rot Problem

Comments drift from code silently. Keep close to code, review during code review, delete rather than let rot.

```python
#  Rotting comment - Code changed, comment didn't
def get_users():
    # Returns active users sorted by name
    return User.query.filter_by(status='active').order_by(User.created_at).all()
    # ↑ Now sorted by created_at, comment lies
```

Docstrings Done Right

```python
#  Wrong - Restates the obvious
def add(a: int, b: int) -> int:
    """Add two integers. Args: a: First integer. b: Second integer."""
    return a + b
```

```python
#  Correct - Documents non-obvious behavior
def calculate_shipping(order: Order) -> Decimal:
    """
    Calculate shipping cost with business rules.

    - Free shipping for orders over $100
    - Hawaii/Alaska adds flat $15 (no free shipping)

    Raises:
        InvalidAddressError: If shipping address is incomplete
    """
```

Relationship to Other Principles

Principle	Connection
Self-Documenting Code	Code shows *what/how*; comments explain *why/why not*
DRY	Don't repeat in comments what the code already says
Single Source of Truth	One authoritative place for each piece of documentation
Boy Scout Rule	Fix stale comments when you touch the code

Summary

1. **Code tells how, comments tell why** — Never explain what code does; explain why it does it
2. **Documentation has layers** — README → API docs → docstrings → inline comments
3. **Comments rot** — Review them during code review; delete rather than let them lie
4. **Anti-patterns abound** — Parrot, journal, and TODO graveyard comments add noise
5. **When in doubt, refactor** — If you need a comment to explain what, the code is unclear

---

Elegance

↑ top

> "Perfection is achieved, not when there is nothing more to add, but

> when there is nothing left to take away." — Antoine de Saint-Exupéry

Core Concept

Beauty through insight. Solves the problem with minimum complexity while revealing something fundamental about the domain.

Four Criteria

Criterion	Description
Minimality	Shortness and simplicity; no superfluous parts
Accomplishment	Does exactly what it should (non-negotiable)
Modesty	Restraint; avoids cleverness and showing off
Revelation	Shows something new about the problem domain

Elegance vs. Cleverness

Elegant Code	Clever Code
Reveals domain insight	Exploits language tricks
Reader says "of course!"	Reader says "how does this work?"
Survives language changes	Implementation-dependent, fragile
Stands alone	Needs explanatory comments

Summary

1. **Minimality** — remove everything superfluous
2. **Accomplishment** — it must work correctly
3. **Modesty** — avoid cleverness and showing off
4. **Revelation** — show insight about the domain
5. **Domain symmetry** — understanding the problem suffices to understand the code

---

Principle of Least Surprise

↑ top

> "In interface design, always do the least surprising thing." — Eric S. Raymond

Core Concept

Components behave as users expect. Never surprise the user.

Strategies

1. **Command-Query Separation**: Separate state-changing methods from queries
2. **Names match behavior**: Naming conventions communicate intent
3. **Consistent return types**: Similar methods return similar types
4. **Sensible defaults**: Most common, safest choice
5. **No hidden side effects**: Methods do only what signatures imply

Common Anti-Patterns

- **Inconsistent Error Handling**: Different methods handle errors differently
- **Misleading Method Names**: Name implies query, actually mutates
- **Surprising Parameter Order**: Non-standard parameter order
- **Spooky Action at a Distance**: Unexpected effects on unrelated parts

Summary

1. **Think like your user**: Design based on what users expect
2. **Separate commands from queries**: Methods that return values shouldn't change state
3. **Names must match behavior**: If you can't name it accurately, the design may be wrong
4. **Consistency over cleverness**: Use established patterns
5. **No hidden side effects**: Every behavior explicit in the signature and name

---

Part II: Architecture

↑ top

> *Structuring and designing systems. These principles govern how code is organized, how components relate, and how systems are designed for change.*

Organization & Structure

↑ top

Where does this code belong? DRY and Single Source of Truth ensure knowledge lives in one place, Separation of Concerns defines boundaries between responsibilities, and Modularity packages those boundaries into self-contained units.

DRY: Don't Repeat Yourself

↑ top

> "Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." — Andy Hunt & Dave Thomas, *The Pragmatic Programmer*

Core Concept

DRY is about **knowledge**, not code. Avoid duplication of *meaning*, not syntax.

Two types: 1. **Knowledge Duplication** — Same business rule in multiple places. **Always fix.** 2. **Incidental Duplication** — Code *looks* similar but represents *different* concepts. **Leave it.** Merging couples unrelated concerns.

The Rule of Three

> **First time**: Write it. **Second time**: Note it. **Third time**: Abstract it.

Patience to find the *right* abstraction. Two occurrences can't distinguish true duplication from incidental similarity. Three reveal the pattern.

The Wrong Abstraction

> "Duplication is far cheaper than the wrong abstraction." — Sandi Metz

The sunk cost trap: Developer A creates an abstraction. Developer B needs similar functionality but not quite—adds a parameter. Developer C adds another. Developer D adds conditionals. Eventually the abstraction becomes incomprehensible, but no one deletes it because of the investment already made.

The fix: When an abstraction accumulates conditionals or parameters to handle "just one more case," inline it back into all callers, delete the unnecessary parts, and start fresh. It's cheaper to re-extract than to maintain the wrong abstraction.

Recognizing True vs. Incidental Duplication

True Knowledge Duplication (FIX)	Incidental Similarity (LEAVE)
Same business rule/concept	Different business concepts
Changes *must* affect all instances	Instances will evolve independently
3+ occurrences confirm the pattern	1-2 occurrences—pattern unclear
Abstraction simplifies	Abstraction requires conditionals

True Knowledge Duplication (FIX)	Incidental Similarity (LEAVE)
Single source of truth needed	Coupling would be harmful

Common Violations

Obvious: Copy-pasted functions, duplicated validation, repeated magic numbers

Hidden: Inconsistent business rules across apps, divergent type definitions, scattered config, parallel data structures (DB columns in SQL strings AND ORM models)

Anti-Patterns

```python
#  Over-DRY: Merged with conditionals
def get_user_by_something(identifier, by_type):
    if by_type == "id": ...
    elif by_type == "email": ...

#  Separate functions with clear responsibilities
def get_user_by_id(user_id: int) -> User: ...
def get_user_by_email(email: str) -> User: ...
```

```python
#  Premature abstraction (Student/Teacher trap)
class Person:
    def get_full_name(self): return f"{self.first} {self.last}"
class Student(Person): pass
class Teacher(Person): pass  # Later needs middle name-abstraction wasted

#  Keep separate until pattern proven
class Student:
    def get_full_name(self): return f"{self.first} {self.last}"
class Teacher:
    def get_full_name(self): return f"{self.first} {self.middle} {self.last}"
```

Refactoring Techniques

Technique	When to Use
Extract Method	Duplicated logic in same class
Extract Class	Duplication spans multiple methods
Extract Superclass	Multiple classes share behavior (Template Method)
Parameterize Method	Methods differ only in values
Composition	Complex inheritance hierarchies

DRY Beyond Code

- **Database**: Define constraints once in schema, not duplicated in app
- **API**: Generate OpenAPI from code (FastAPI/Pydantic), don't maintain separately
- **Config**: Centralize in one module, import everywhere
- **Docs**: Single source of truth, reference elsewhere
- **Infrastructure**: Similar infrastructure components may warrant deduplication.

Summary

1. **Knowledge duplication is always a code smell**—always fix it
2. **Incidental similarity is not duplication**—don't merge different concepts
3. **Rule of Three**: Patience to find the *right* abstraction, not permission to ignore duplication
4. **Wrong abstractions**: Delete and start over—they merged incidental similarity
5. **Beyond code**: Databases, APIs, config, documentation (see also: Single Source of Truth)

Single Source of Truth

↑ top

> "There should be one—and preferably only one—obvious way to store a piece of information."

Core Concept

Every piece of data has exactly one authoritative location. All other references derive from that source. The problem: when data exists in multiple places, which is correct?

SSoT vs. DRY

Aspect	DRY	SSoT
Focus	Code and logic duplication	Data storage duplication
Scope	Within a codebase	Across systems and databases
Violation	Copy-pasted functions	Same field in multiple tables

Aspect	DRY	SSoT
Fix	Extract to shared function	Designate authoritative source

Common Violations

1. **Storing Foreign Keys in Multiple Databases**
2. **Duplicating User Data Across Services**
3. **Storing Derived Data Without Clear Ownership**

When Duplication Is Acceptable

1. **Intentional Caching** with TTL
2. **Read Model Denormalization** (CQRS)
3. **Computed/Derived Values**
4. **Cross-Region Replication**

Summary

1. **Every piece of data needs exactly one authoritative source**
2. **Other systems should reference, not duplicate** authoritative data
3. **Derived/computed values are acceptable** — they don't need synchronization
4. **Ask "which is correct?"** — if you can't answer immediately, fix the design

---

Separation of Concerns

↑ top

> "The separation of concerns, even if not perfectly possible, is yet the only available technique for effective ordering of one's thoughts." — Edsger W. Dijkstra

Core Concept

Decompose systems into distinct parts, each addressing one concern. A "concern" = any aspect of functionality (business logic, persistence, UI, etc.).

Measures: High cohesion (related things together) · Low coupling (unrelated things independent)

Types of Concerns

Type	Examples
Functional	Authentication, data processing, payment
Non-functional	Performance, security, scalability
Cross-cutting	Logging, error handling, caching

Common Violations

Code Smells: DB queries in UI handlers, business rules in CSS, validation scattered across layers, formatting in business classes.

SoC-Specific Anti-Patterns:

Anti-Pattern	Description	Fix
Blob/God Object	One class centralizes most functionality	Split into single-purpose classes
Divergent Change	One class changes for multiple reasons	Extract class per reason
Shotgun Surgery	One change modifies many places	Consolidate related logic

Anti-Patterns

```python
#  Wrong - Mixed concerns: business logic + presentation + I/O
def process_order(order_id):
    order = db.query(f"SELECT * FROM orders WHERE id = {order_id}")
    if order.total > 100:
        order.discount = order.total * 0.1
    print(f"<div class='order'>Order #{order.id}: ${order.total}</div>")
    send_email(order.customer, "Your order is ready")

#  Correct - Separated concerns
class OrderRepository:
    def get_by_id(self, order_id: int) -> Order:
        return self.db.query(Order).get(order_id)

class OrderService:
    def apply_discount(self, order: Order) -> Order:
        if order.total > 100:
            order.discount = order.total * 0.1
        return order
```

```python
class OrderPresenter:
    def to_html(self, order: Order) -> str:
        return f"<div class='order'>Order #{order.id}: ${order.total}</div>"
```

Summary

1. **One concern per component** — functions, classes, modules, layers
2. **High cohesion, low coupling** — related together, unrelated separate
3. **Natural boundaries** — separate where concerns genuinely differ
4. **Avoid over-separation** — don't fragment for its own sake

---

Modularity

↑ top

> "Every module is characterized by its knowledge of a design decision which it hides from all others." — David Parnas

Core Concept

Modularity is **dividing software into independent components** where each module encapsulates a specific responsibility and hides implementation details behind a well-defined interface.

The Parnas Principle: Decompose systems by **design decisions likely to change**. Each module hides one decision.

Two measures: 1. **Cohesion** — How strongly elements within a module belong together (aim: high) 2. **Coupling** — How much modules depend on each other's internals (aim: low)

Deep vs. Shallow Modules

Type	Characteristics
Deep	Simple interface, complex implementation
Shallow	Complex interface, little hidden

Aim for depth: Hide significant complexity behind minimal APIs.

Common Violations

Code Smells: God Class, Feature Envy, Shotgun Surgery, Utilities junk drawer

Summary

1. **Hide design decisions** — Each module encapsulates one decision likely to change
2. **High cohesion** — Elements within a module belong together
3. **Low coupling** — Modules depend only on interfaces
4. **Deep over shallow** — Simple interface, complex implementation
5. **No God modules** — If it does "everything," it encapsulates nothing

---

Coupling & Dependencies

↑ top

How do components relate to each other? These principles minimize unhealthy dependencies. Encapsulation hides internal state, Law of Demeter limits knowledge of other objects, Orthogonality ensures independent change, Dependency Injection makes dependencies explicit, and Composition Over Inheritance favors flexible composition.

Encapsulation

↑ top

> "Ask not what an object knows; ask what it can do for you."

Core Concept

Bundle data with behavior, hide internals behind interfaces.

1. **Bundling**: Group related data and behavior
2. **Information Hiding**: Restrict direct access to internal state

Tell, Don't Ask

Don't query state and decide externally—tell the object what to do.

```
#  Wrong - Asking for state, making decisions externally
def process_order(order):
    if order.get_status() == "pending":
        if order.get_total() > 100:
            discount = order.get_total() * 0.1
            order.set_total(order.get_total() - discount)
        order.set_status("processed")

#  Correct - Telling the object what to do
```

```python
def process_order(order):
    order.process()  # Order knows its own business rules
```

Common Violations

- **Data Classes Without Behavior**: A "data class" that only contains fields and getters/setters
- **Getter/Setter Pairs That Add No Value**: Accessors without validation or computation
- **Returning Mutable Internal State**: Allowing callers to corrupt object invariants
- **Feature Envy**: Methods that use more data from another class than their own

Summary

1. **Bundle data with behavior** — Objects should do things, not just hold data
2. **Hide implementation details** — Internals can change without affecting callers
3. **Tell, don't ask** — Command objects to act rather than querying their state
4. **Protect invariants** — Use access control to enforce object validity (see also: Parse, Don't Validate)

Law of Demeter

↑ top

> "Each unit should have only limited knowledge about other units: only talk to your immediate friends; don't talk to strangers." — Ian Holland

Core Concept

Limit knowledge of other objects' structure. Only interact with immediate dependencies, not through them.

The "One Dot" Rule

```python
#  Wrong - Multiple dots (train wreck)
customer.get_wallet().get_credit_card().charge(amount)

#  Correct - One dot
customer.charge(amount)  # Customer knows how to charge itself
```

Formal Definition

A method `m` of object `a` may only invoke methods of: - `a` itself - `m`'s parameters - Objects created within `m` - `a`'s direct attributes - Global/module-level objects

Forbidden: Methods of objects returned by other method calls.

Exceptions: When Chaining Is Acceptable

Pattern	Why It's OK
Builder pattern	Same object returned; configures self
Fluent interfaces	Designed for chaining; returns `self`
Data Transfer Objects	No behavior to encapsulate
Standard library	`"hello".strip().upper()` — string ops

A Note on Tell-Don't-Ask

> "Tell-Don't-Ask encourages moving behavior into objects, but don't become a Getter Eradicator." — Martin Fowler

Objects sometimes collaborate effectively by *providing* information. Transformers that simplify data for clients (like `EmbeddedDocument`) are valid query methods. The principle is about co-locating behavior with data, not eliminating all accessors.

Summary

1. **Only talk to immediate friends** — don't reach through objects
2. **One dot rule** — `a.b()` good, `a.b().c()` suspect
3. **Tell, don't ask** — command objects, don't interrogate (see also: Encapsulation)
4. **Exceptions exist** — builders, fluent APIs, DTOs are fine

---

Orthogonality

↑ top

> "Eliminate effects between unrelated things. Design self-contained components: independent, and with a single, well-defined purpose." — Andy Hunt & Dave Thomas

Core Concept

Changes in one component don't affect others. Like a helicopter with coupled controls: fix one bug, two more pop up elsewhere.

Common Violations

- **Global State**: Becomes a coupling point between different parts
- **Database-Coupled Business Logic**: SQL dialects leak into business logic
- **Presentation Mixed with Logic**: Changing display requires changing computation
- **Feature Creep in Objects**: Objects accumulate responsibilities

Summary

1. **Two components are orthogonal if changes in one don't affect the other**
2. **Coupling is viral** — a little leads to more
3. **Measure orthogonality** by how many places change when one requirement changes
4. **Techniques that help**: dependency injection, abstract interfaces, avoiding global state

---

Dependency Injection

↑ top

> "The key benefit of Dependency Injection is that it removes the dependency that a class has on a concrete implementation." — Martin Fowler

Core Concept

Dependencies "injected" from outside rather than created internally. A class declares what it needs, not how to get it.

Three Forms

1. **Constructor Injection** (Preferred): Through constructor
2. **Setter Injection**: Through setters after construction
3. **Interface Injection**: Dependency provides injector method

Anti-Patterns

```
#  Wrong - Hardcoded dependency
class MovieLister:
    def __init__(self):
        self._finder = ColonDelimitedMovieFinder("movies.txt")  # Coupled!

#  Correct - Injected dependency
```

```python
class MovieLister:
    def __init__(self, finder: MovieFinder):
        self._finder = finder
```

Service Lifetimes

Lifetime	Instance Created	Use Case
Transient	Every time requested	Lightweight, stateless services
Scoped	Once per scope/request	Request-specific state
Singleton	Once for application lifetime	Expensive to create, shared state

Summary

1. **DI decouples classes from dependencies** — clients declare needs, not solutions
2. **Constructor injection is preferred** — explicit, immutable, testable
3. **Too many dependencies = SRP violation** — DI makes this visible
4. **Testability is the primary benefit** — swap real dependencies for test doubles

---

Composition Over Inheritance

↑ top

> "Favor object composition over class inheritance." — Gang of Four, *Design Patterns*

Core Concept

Build complex behavior by combining objects rather than extending classes.

- **Inheritance** ("is-a"): White-box — subclass sees parent internals
- **Composition** ("has-a"): Black-box — interact via interfaces only

Why Composition Is Preferred

Inheritance Problem	Composition Solution
Tight coupling to parent	Loose coupling via interfaces
Changes cascade to subclasses	Changes isolated to components

Inheritance Problem	Composition Solution
Hierarchy fixed at compile-time	Components swappable at runtime
Class explosion for combinations	Mix components as needed
Fragile base class problem	No inherited implementation details

Anti-Patterns

```python
#  Wrong - Class explosion via inheritance
class FileLogger: ...
class FileLoggerWithEncryption(FileLogger): ...
class FileLoggerWithCompression(FileLogger): ...
# Combinatorial explosion!

#  Correct - Composition
class Logger:
    def __init__(self, writer: Writer, filters: list[Filter]):
        self.writer = writer
        self.filters = filters

logger = Logger(FileWriter(), [EncryptionFilter(), CompressionFilter()])
```

Summary

1. **Composition = "has-a"**, Inheritance = "is-a" — choose appropriately
2. **Inheritance breaks encapsulation** — changes cascade unpredictably
3. **Class explosion** — composition avoids combinatorial hierarchies
4. **Runtime flexibility** — swap components without recompiling

---

Design Patterns & Conventions

↑ top

Proven approaches to common problems. SOLID provides five foundational OO principles, Convention Over Configuration reduces boilerplate through sensible defaults, Command-Query Separation distinguishes actions from queries, and Code Reusability addresses when and how to make code reusable.

SOLID Principles

↑ top

> "SOLID principles are the foundation of good software design—they make code more maintainable, flexible, and testable." — Robert C. Martin (Uncle Bob)

Overview

Letter	Principle	Core Idea
S	Single Responsibility	One reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes must be substitutable for base types
I	Interface Segregation	Many specific interfaces over one general
D	Dependency Inversion	Depend on abstractions, not concretions

S — Single Responsibility Principle

> "A class should have one, and only one, reason to change."

Violations: Mixed I/O and logic, persistence in domain objects, god classes, class names with "And" or "Manager"

O — Open/Closed Principle

> "Software entities should be open for extension but closed for modification."

Violations: `if/elif` chains checking types, `isinstance()` checks, modifying existing code for new variants

L — Liskov Substitution Principle

> "Subtypes must be substitutable for their base types."

Violations: Subclass raises `NotImplementedError`, empty `pass` overrides, type checks before method calls

I — Interface Segregation Principle

> "Clients should not be forced to depend on interfaces they do not use."

Violations: Fat interfaces (20+ methods), `raise NotImplementedError` in implementations

D — Dependency Inversion Principle

> "High-level modules should not depend on low-level modules. Both should depend on abstractions."

Violations: Direct instantiation in constructors, concrete imports in business logic, can't mock for testing

When NOT to Apply SOLID

1. **Simple scripts**: Overhead outweighs benefits
2. **Prototyping**: Flexibility over structure
3. **Performance-critical paths**: Abstractions add indirection
4. **Single implementations**: Don't create interfaces for classes that won't have alternatives
5. **Early development**: Wait for patterns to emerge (Rule of Three)

Detection Checklist

Principle	Code Smells
SRP	Class name has "And"/"Manager", methods don't use most attributes
OCP	Adding features requires modifying existing classes, `isinstance()` chains
LSP	Subclass raises `NotImplementedError`, empty overrides, type checks
ISP	Interfaces with 10+ methods, classes implement unused methods
DIP	Direct instantiation in constructors, can't mock for testing

---

Convention Over Configuration

↑ top

> "You're not a beautiful and unique snowflake. By giving up vain individuality, you can leapfrog the toils of mundane decisions, and make faster progress in areas that really matter." — David Heinemeier Hansson, The Rails Doctrine

Core Concept

Sensible defaults that work out of the box. Explicit configuration only when deviating from norm. Core insight: most decisions aren't worth making—if 90% use `id` as primary key, don't force specification.

The Power of Defaults

Without CoC	With CoC
Specify database table name for every model	`User` class → `users` table automatically
Configure primary key column	`id` assumed unless overridden
Define foreign key naming	`user_id` derived from `User` association
Set up file locations manually	`app/models/`, `app/views/`, etc. by convention

Conventions compose: `has_many :posts` resolves `Post` → `posts` table → `user_id` FK automatically.

Real-World Examples

Framework	Convention	Override When Needed
Rails	`User` → `users` table	`self.table_name = "legacy_accounts"`
Spring Boot	Auto-configure from classpath	`@Configuration` for custom beans
Django	`model_name` → `appname_modelname` table	`class Meta: db_table = "custom"`
Next.js	`pages/about.js` → `/about` route	Custom routing configuration
pytest	`test_*.py` files auto-discovered	`pytest.ini` for custom patterns

When to Apply

Good Fit	Poor Fit
Repeated patterns across projects	Highly unique domain requirements
Reducing boilerplate for common cases	Legacy systems with established conventions
Framework/library design	When explicitness aids understanding
Lowering barriers for beginners	Security-critical configurations

Common Violations

```
#  Wrong - Forcing configuration for obvious defaults
class UserService:
```

```python
    def __init__(
        self,
        table_name: str,
        id_column: str,
        created_at_column: str,
        updated_at_column: str,
    ):
        self.table_name = table_name
        self.id_column = id_column
        # ... exhausting

# Usage requires specifying everything
service = UserService(
    table_name="users",
    id_column="id",
    created_at_column="created_at",
    updated_at_column="updated_at"
)

#  Correct - Sensible defaults with escape hatches
class UserService:
    def __init__(
        self,
        table_name: str = "users",
        id_column: str = "id",
        timestamps: bool = True,
    ):
        self.table_name = table_name
        self.id_column = id_column
        self.timestamps = timestamps

# Usage: zero config for common case
service = UserService()  # Just works

# Override only what differs
legacy_service = UserService(table_name="legacy_accounts")
```

The Dark Side

1. **Hidden Magic** — implicit behavior hard to debug
2. **Learning Cliff** — must learn convention to deviate
3. **Rigidity at Scale** — common-case optimizations may not scale

```python
# When convention fails: trying to use a legacy database
# Rails convention: User → users table
# Reality: Legacy DB uses "tbl_usr_accounts"
```

```
# Now you're fighting the framework instead of working with it
```

Explicit vs. Implicit Trade-off

Approach	Advantages	Disadvantages
Explicit (Configuration)	Clear, searchable, no surprises	Verbose, repetitive, decision fatigue
Implicit (Convention)	Concise, consistent, fast start	Hidden behavior, learning curve

Python's "Explicit > implicit" seems contradictory. Resolution: conventions must be *discoverable* and well-documented.

Relationship to Other Principles

Principle	Relationship
KISS	Both reduce unnecessary complexity; CoC removes decision complexity
YAGNI	Don't configure what you don't need to configure
DRY	Conventions eliminate repetitive configuration
Cognitive Load	Fewer decisions = lower mental burden
Principle of Least Surprise	Good conventions match developer expectations

Summary

1. **Provide sensible defaults** — Common cases should require zero configuration
2. **Allow overrides** — Escape hatches for when convention doesn't fit
3. **Conventions compose** — Build deeper abstractions from consistent patterns
4. **Document the magic** — Implicit behavior must be discoverable
5. **Know when to deviate** — Convention serves you until it doesn't; then configure explicitly

---

Command-Query Separation

↑ top

> "Asking a question should not change the answer." — Bertrand Meyer

Core Concept

Type	Purpose	Returns	Side Effects
Query	Return info	Yes	None
Command	Change state	None	Yes

Methods returning values shouldn't change state. Methods changing state shouldn't return values.

Why CQS Matters

1. **Reasoning Confidence**: Queries are safe to call anywhere
2. **Testing Simplicity**: Queries tested in isolation
3. **Caching Safety**: Queries can be cached
4. **Parallelization**: Queries run concurrently without race conditions

Anti-Patterns

```python
#  Wrong - Modifies AND returns
def get_or_create_user(self, email: str) -> User:
    user = self.db.find_by_email(email)
    if not user:
        user = User(email=email)
        self.db.save(user)  # Side effect!
    return user

#  Correct - Separate operations
def find_user_by_email(self, email: str) -> User | None:
    """Query: Returns user or None, no side effects."""
    return self.db.find_by_email(email)

def create_user(self, email: str) -> None:
    """Command: Creates user, returns nothing."""
    self.db.save(User(email=email))
```

Pragmatic Exceptions

- Stack pop operation (atomic)
- Thread-safe increment-and-get
- Database identity generation

Summary

1. **Separate queries from commands** — Return value OR change state, not both

2. **Queries are safe** — Call them anywhere, cache them, parallelize them
3. **Commands need care** — Order matters, test state changes explicitly
4. **Break CQS pragmatically** — Atomic operations sometimes require both

---

Code Reusability

↑ top

> "A little copying is better than a little dependency." — Rob Pike

Core Concept

Code usable in multiple contexts without modification. Unlike DRY (eliminating existing duplication), reusability is forward-looking.

The paradox: Reusable components cost 3-10x more to develop. Payoff only materializes with actual reuse.

Characteristics of Reusable Code

Trait	Description
Modular	Self-contained with minimal external dependencies
Generic	Handles a range of inputs without modification
Well-documented	Clear API, usage examples, edge cases documented
Stable Interface	Public API changes infrequently
Thoroughly Tested	Works reliably across scenarios

Types of Reuse

Type	Scope	Example
Copy-paste	Lowest	Snippets, templates
Functions	Local	Utility functions within a project
Libraries	Organization	Shared packages across teams
Frameworks	Industry	Django, React, Rails

The Reusability Trap

Designing for reuse before proving need creates complexity without value. Rule of Three applies: wait until three different contexts.

Santa Claus Problem: A billion open source components—finding, learning, and integrating often costs more than the reuse saves.

```python
#  Wrong - Premature reusability (YAGNI violation)
class GenericDataProcessor:
    """Handles any data format with any transformation."""
    def __init__(self, parser, transformer, validator, serializer):
        self.parser = parser
        self.transformer = transformer
        self.validator = validator
        self.serializer = serializer

    def process(self, data, options=None):
        options = options or {}
        parsed = self.parser.parse(data, **options.get('parse', {}))
        transformed = self.transformer.transform(parsed, **options.get('transform', {}))
        if options.get('validate', True):
            self.validator.validate(transformed)
        return self.serializer.serialize(transformed, **options.get('serialize', {}))

#  Correct - Start specific, generalize when needed
def parse_user_csv(csv_data: str) -> list[dict]:
    """Parse user data from CSV format."""
    rows = csv_data.strip().split('\n')
    headers = rows[0].split(',')
    return [dict(zip(headers, row.split(','))) for row in rows[1:]]
```

Designing for Reusability

When code has proven its need for reuse, apply these principles:

1. Minimize Dependencies

```python
#  Wrong - Tight coupling to specific libraries
def format_date(date):
    import pandas as pd  # Heavy dependency for simple task
    return pd.Timestamp(date).strftime('%Y-%m-%d')

#  Correct - Use standard library
from datetime import datetime

def format_date(date: datetime) -> str:
    return date.strftime('%Y-%m-%d')
```

2. Accept Abstract Inputs

```python
#  Wrong - Only accepts specific type
def process_users(users: list[User]) -> None:
```

```python
    for user in users:
        send_email(user.email)

#  Correct - Accept any iterable of objects with email
from typing import Protocol, Iterable

class HasEmail(Protocol):
    email: str

def process_contacts(contacts: Iterable[HasEmail]) -> None:
    for contact in contacts:
        send_email(contact.email)
```

3. Provide Sensible Defaults

```python
#  Wrong - Requires all parameters
def retry(func, max_retries, delay, backoff_factor, exceptions):
    ...

#  Correct - Sensible defaults, only specify what differs
def retry(
    func,
    max_retries: int = 3,
    delay: float = 1.0,
    backoff_factor: float = 2.0,
    exceptions: tuple = (Exception,),
):
    ...
```

Common Violations

Code Smells: - Over-parameterized functions trying to handle every case - Components that can't be tested in isolation - Libraries that require complex configuration before basic use - Code with implicit dependencies on global state

Organizational Barriers: - Politics: Teams block other teams from using "their" code - Psychology: Developers view reuse as stifling creativity - NIH Syndrome: "Not Invented Here" bias against external solutions

When Reusability Hurts

Verbose, redundant code sometimes beats elegant abstractions: - **Debugging**: Isolated code means problems stay isolated - **Onboarding**: Simple duplication is easier to understand than clever abstractions - **Change velocity**: Modifying copy-pasted code can't break other systems - **Coupling**: "Reusable" components become coupling points across systems

The construction paradox: demolishing and rebuilding often costs less than

renovating. Similarly, rewriting 50 lines sometimes beats understanding 500 lines of "reusable" framework code.

Summary

1. **Reusability is earned, not designed** — Wait for three use cases before investing
2. **Upfront cost is real** — Reusable code costs more to develop and understand
3. **Dependencies are the enemy** — Minimize external coupling; a little copying beats a little dependency
4. **Simple duplication can be better** — Isolated, obvious code often beats clever abstractions
5. **Stable interfaces enable reuse** — Public APIs should change rarely

---

Data & State

↑ top

How should data flow and behave? Parse Don't Validate transforms unstructured input into typed domain objects at boundaries. Immutability eliminates bugs by preventing state changes after creation. Idempotency ensures operations can be safely repeated.

Parse, Don't Validate

↑ top

> "A parser is just a function that consumes less-structured input and produces more-structured output." — Alexis King

Core Concept

Transform data into precise types that make illegal states unrepresentable. Validation checks then forgets. Parsing checks and *remembers* in the type system.

Validation vs. Parsing

```
#  Wrong - Validation: checks then discards knowledge
def validate_non_empty(items: list) -> None:
    if not items:
        raise ValueError("List cannot be empty")
    # Returns nothing-knowledge is lost
```

```python
def process(items: list) -> None:
    validate_non_empty(items)
    first = items[0]  # Caller must trust validation happened

#  Correct - Parsing: checks and returns proof
from typing import NewType, TypeVar
T = TypeVar('T')
NonEmptyList = NewType('NonEmptyList', list)

def parse_non_empty(items: list[T]) -> NonEmptyList[T]:
    if not items:
        raise ValueError("List cannot be empty")
    return NonEmptyList(items)  # Type proves non-emptiness

def process(items: NonEmptyList[T]) -> None:
    first = items[0]  # Type guarantees safety-no trust needed
```

The Shotgun Parsing Anti-Pattern

Checks spread everywhere hoping to catch bad data: 1. **Redundant checks**: Same validation repeated 2. **Inconsistent coverage**: Easy to miss checks 3. **Rollback hell**: Invalid data after partial processing 4. **Silent corruption**: Invalid state if check forgotten

```python
#  Wrong - Shotgun parsing
def get_user(user_id: str) -> User:
    if not user_id:
        raise ValueError("user_id required")
    ...

def update_user(user_id: str, data: dict) -> None:
    if not user_id:  # Repeated check!
        raise ValueError("user_id required")
    ...

#  Correct - Parse once at the boundary
UserId = NewType('UserId', str)

def parse_user_id(raw: str) -> UserId:
    if not raw or not raw.strip():
        raise ValueError("user_id required")
    return UserId(raw.strip())

def get_user(user_id: UserId) -> User: ...      # No validation needed
def update_user(user_id: UserId, data: dict): ...  # Type guarantees validity
```

Primitive Obsession

Over-reliance on `str`, `int`, `dict` for domain concepts. Primitives carry no context—validation knowledge is lost.

```
#  Wrong - Primitive obsession
def create_order(customer_id: str, product_id: str, quantity: int, price: float): ...
# Easy to swap customer_id/product_id; negative quantity allowed; what currency?

#  Correct - Domain types encode constraints
def create_order(customer_id: CustomerId, product_id: ProductId,
                 quantity: PositiveInt, price: Money): ...
```

Make Illegal States Unrepresentable

```
#  Wrong - Invalid states representable
@dataclass
class Order:
    status: str  # "pending", "shipped", "delivered"
    shipped_at: datetime | None  # Bug: can be None when status="shipped"

#  Correct - Invalid states unrepresentable
@dataclass
class PendingOrder:
    items: list[Item]

@dataclass
class ShippedOrder:
    items: list[Item]
    shipped_at: datetime  # Required-impossible to forget

Order = PendingOrder | ShippedOrder | DeliveredOrder
```

Parse at the Boundary

```
#  Wrong - Raw data flows through system
def handle_request(request: dict) -> Response:
    user_id = request.get("user_id")
    if not user_id:
        raise ValueError("user_id required")
    # More validation scattered deeper...

#  Correct - Parse at boundary, use typed data internally
@dataclass(frozen=True)
class CreateUserRequest:
    user_id: UserId
    email: Email
```

```
    age: PositiveInt

def parse_request(raw: dict) -> CreateUserRequest:
    return CreateUserRequest(
        user_id=parse_user_id(raw.get("user_id", "")),
        email=parse_email(raw.get("email", "")),
        age=parse_positive_int(raw.get("age", 0)),
    )

def handle_request(request: CreateUserRequest) -> Response:
    ...  # All data already validated
```

Lightweight Parsing with NewType

`NewType` marks validated data without runtime overhead:

```
from typing import NewType

UserId = NewType('UserId', str)  # Still a str at runtime

def parse_user_id(raw: str) -> UserId:
    if not raw or not raw.startswith("U"):
        raise ValueError("Invalid user ID")
    return UserId(raw)

def load_user(user_id: UserId) -> User: ...

load_user(parse_user_id(url))  #  OK
load_user("U6789679")  #  Type checker error
```

Pydantic: Full-Throttle Parsing

```
from pydantic import BaseModel, TypeAdapter

class User(BaseModel):
    username: str
    age: int  # Coerced from string automatically
    last_login: datetime

users = TypeAdapter(list[User]).validate_json(raw_json)
```

Common Violations

- **Functions returning `None` after validation** — Return the proof instead
- **Boolean flags instead of types** — `is_valid: bool` vs. `ValidatedData` type
- **Re-validating inside trusted code** — Parse at boundaries only

- **Passing raw dicts through layers** — Parse to domain types at the edge
- **Using `str` for everything** — Email, phone, SSN as `str` is primitive obsession

When NOT to Apply

- **Quick scripts**: Overhead of custom types may not pay off
- **Performance-critical paths**: Sometimes primitives are faster
- **Prototyping**: Over-engineering types slows exploration
- **Simple CRUD**: Not every field needs a custom type

Summary

1. **Parsers return proof, validators return nothing** — Transform data into types that encode validity
2. **Parse at the boundary** — Convert external data to domain types immediately
3. **Make illegal states unrepresentable** — Design types where invalid combinations can't exist
4. **Eliminate shotgun parsing** — Centralize validation, then trust the types
5. **Avoid primitive obsession** — Use domain types instead of raw `str`/`int`
6. **Choose parsing depth** — `NewType` for lightweight, classes for rich, Pydantic for full (see also: Fail-Fast, Design by Contract, Encapsulation, Immutability)

Immutability

↑ top

> "Immutable types are safer from bugs, easier to understand, and more ready for change." — MIT 6.005 Software Construction

Core Concept

Once created, state cannot be modified. Create new structures with desired changes instead.

Mutable shared state causes most concurrency and aliasing bugs. Immutability eliminates them by design.

Benefits

- **Thread safety without locks**: Share freely between threads
- **No defensive copying**: Share directly

- **Simpler reasoning**: Only understand creation site
- **Safe hash keys**: Can be dictionary keys
- **Enables caching**: Results remain valid indefinitely

Common Violations

```
#  Wrong - Mutates caller's data
def normalize_scores(scores: list[float]) -> list[float]:
    for i in range(len(scores)):
        scores[i] /= max(scores)  # Mutates the input!
    return scores

#  Correct - Returns new list
def normalize_scores(scores: list[float]) -> list[float]:
    max_score = max(scores)
    return [score / max_score for score in scores]
```

Python Implementation

```
from dataclasses import dataclass

#  Immutable dataclass
@dataclass(frozen=True)
class Document:
    gdrive_id: str
    file_name: str
    content_hash: str

#  Use tuple instead of list for fixed data
SUPPORTED_EXTENSIONS: tuple[str, ...] = (".pdf", ".docx", ".txt")

#  Use frozenset instead of set
VALID_STATUSES: frozenset[str] = frozenset({"pending", "done", "failed"})
```

Summary

1. **Immutable objects can't change** — once created, their value is fixed
2. **Aliasing is safe** with immutable objects
3. **Thread safety is free** — no locks needed
4. **Prefer immutability** — use frozen dataclasses, tuples, frozensets (see also: Parse, Don't Validate)

Idempotency

↑ top

> "An operation is idempotent if performing it multiple times has the same effect as performing it once."

Core Concept

Multiple executions produce same result as one. In distributed systems, duplicate requests are inevitable—design around them.

Implementation Strategies

1. **Idempotency Keys**: Attach unique identifier to each request
2. **Deterministic IDs**: Generate IDs from content itself
3. **Database Upserts**: Use `INSERT ... ON CONFLICT`
4. **Conditional Writes**: Use version numbers (optimistic locking)
5. **Lease-Based Processing**: Acquire exclusive access before processing

Naturally Idempotent Operations

Operation	Why Idempotent
`GET /resource`	Reads don't change state
`PUT /resource`	Full replacement, same result
`DELETE /resource`	Deleting twice = still deleted
Setting a value	`x = 5` is idempotent; `x += 5` is not

Summary

1. **Duplicates are inevitable** in distributed systems—design for them
2. **Use deterministic IDs** derived from content when possible
3. **Prefer upserts** over inserts for database operations
4. **Track processed messages** in queue consumers
5. **Test by calling twice** and verifying same result

Part III: Reliability

↑ top

> *Building robust, maintainable systems. These principles govern how code handles errors, maintains itself over time, and operates in production.*

Robustness & Safety

↑ top

How does code handle the unexpected? Fail-Fast detects errors early, Design by Contract makes expectations explicit, Postel's Law enables interoperability, Resilience keeps systems running despite failures, and Principle of Least Privilege limits damage from breaches.

Fail-Fast & Defensive Programming

↑ top

> "The best debugging is the debugging you never have to do because you found the problem immediately." — Jim Shore

Core Concept

Detect and report errors at the earliest possible moment. Don't let invalid state propagate.

1. **Fail-Fast** — Detect early, fail immediately with clear diagnostics
2. **Defensive Programming** — Anticipate misuse, validate at boundaries

Design by Contract

↑ top

Contract	Responsibility	Example
Preconditions	Caller must satisfy before calling	`assert user_id is not None`
Postconditions	Method must satisfy before returning	`assert result.is_valid()`
Invariants	Must hold throughout object lifetime	`assert self.balance >= 0`

Common Patterns

```python
#  Guard Clauses - Fail fast at entry
def process_order(order):
    if order is None:
        raise ValueError("order required")
    if not order.items:
        raise ValueError("items required")
```

```python
#  Config Validation at Startup
def __init__(self):
    self.key = os.getenv("API_KEY")
    if not self.key:
        raise ConfigError("API_KEY required")
```

Error Handling Strategies

Error Type	Strategy
Precondition violation	Raise immediately
Transient failure	Retry with backoff
Deterministic failure	Fail permanently
Invariant violation	Assert (crash in dev)

Summary

1. **Validate early** — Check inputs at function entry, config at startup
2. **Fail loudly** — Clear error messages beat silent corruption
3. **Distinguish error types** — Transient (retry) vs. deterministic (fail) vs. bug (crash)
4. **Use assertions for invariants** — Things that should never be false (see also: Design by Contract)
5. **Trust validated data** — Don't re-validate inside trusted boundaries (see also: Parse, Don't Validate)

Design by Contract

↑ top

> "A software system is not a bunch of components thrown together. It is a construction of interacting elements, connected by clear contracts." — Bertrand Meyer

Core Concept

Agreements between callers and routines. Functions promise results (postconditions) **if** callers meet requirements (preconditions).

The Three Pillars

Element	Definition	Who Benefits	Who Obligates
Precondition	What must be true before	Supplier	Client

Element	Definition	Who Benefits	Who Obligates
Postcondition	What the routine guarantees	Client	Supplier
Invariant	What must always be true	Both	Supplier

Inheritance Rules (Liskov Substitution)

Contract Element	Subtype Rule
Preconditions	Can only be **weakened**
Postconditions	Can only be **strengthened**
Invariants	Can only be **strengthened**

DbC vs. Defensive Programming

Aspect	Design by Contract	Defensive Programming
Philosophy	Trust but verify at boundaries	Trust no one
Responsibility	Caller ensures preconditions	Callee handles all cases
When to use	Internal interfaces	External interfaces

Summary

1. **Contracts make responsibilities explicit** — Caller ensures preconditions; supplier ensures postconditions
2. **Invariants define valid object state** — Must hold after construction and every public method
3. **Assertions are executable contracts** — Document and verify simultaneously (see also: Fail-Fast)
4. **DbC complements defensive programming** — Use DbC internally, defensive at boundaries

---

Postel's Law (Robustness Principle)

↑ top

> "Be conservative in what you send, be liberal in what you accept."
> — Jon Postel, RFC 793 (1981)

Core Concept

Conservative output, liberal input. Generate strictly conformant output; accept non-conformant input if meaning is clear. Instrumental in Internet's growth.

- **Conservative output** — Follow specs exactly
- **Liberal input** — Accept reasonable variations

Real-World Examples

System	How Postel's Law Applied	Outcome
HTML Browsers	Render malformed HTML gracefully	Web grew explosively; browser code became nightmarishly complex
Unix Pipes	Tools accept varied input, produce consistent output	Composable ecosystem; `cat`, `grep`, `sort` chain reliably
SMTP Email	Accept lines >998 chars despite spec limit	Pragmatic interop; many systems now depend on non-standard behavior
JSON APIs	Ignore unknown fields	Forward-compatible evolution; old clients work with new servers

When to Apply

Context	Recommendation
Protocol extensions	Ignore unknown fields; don't reject
Public APIs	Accept variations; can't control all clients
Backward compatibility	Old clients shouldn't break with new servers
Security-critical input	Validate strictly; liberal acceptance creates attack surface
Internal systems	Strict validation catches bugs early

The Tolerant Reader Pattern

Ignore unknown fields rather than failing:

```
#  Wrong - Strict parsing breaks when API adds fields
def parse_user(data: dict) -> User:
    if set(data.keys()) != {"id", "name", "email"}:
        raise ValueError("Unexpected fields in response")
    return User(id=data["id"], name=data["name"], email=data["email"])

#  Correct - Tolerant reader ignores unknown fields
def parse_user(data: dict) -> User:
    return User(
        id=data["id"],       # Required: fail if missing
        name=data["name"],   # Required: fail if missing
        email=data["email"]  # Required: fail if missing
        # Unknown fields silently ignored - forward compatible
    )
```

The Dark Side

Postel's Law has significant criticisms in modern hostile environments:

Problem	Description
Specification Rot	Receivers accept malformed input → senders never fix bugs → incorrect behavior becomes de facto standard
Security Vulnerabilities	"Reasonable" input may be crafted to exploit edge cases
Hidden Bugs	Liberal receivers mask sender bugs; problems surface years later
Bug-for-Bug Compatibility	New implementations must replicate bugs to maintain compatibility

HTML Lesson: Browser tolerance enabled rapid growth but created nightmares—"incorrect" became the only way.

Modern Balanced Approach

```
#  Balance: strict where it matters, tolerant for extensibility
def process_webhook(data: dict) -> None:
    # STRICT: Validate required fields (fail-fast)
    if "event_type" not in data:
        raise ValueError("Missing required field: event_type")
    if "timestamp" not in data:
        raise ValueError("Missing required field: timestamp")
```

```
# TOLERANT: Ignore unknown fields (enables future extensions)
event_type = data["event_type"]
payload = data.get("payload", {})

# CONSERVATIVE: Output follows strict contract
handle_event(event_type, payload)
```

Relationship to Other Principles

Principle	Relationship
Fail-Fast	Tension: liberal acceptance delays failure detection; balance by validating *required* fields strictly
Resilience	Supportive: liberal acceptance aids graceful degradation
Principle of Least Surprise	Supportive: conservative output is predictable
Defensive Programming	Tension: strict boundary validation vs. liberal acceptance

Summary

1. **Conservative output, liberal input** — Generate strictly, accept generously
2. **Enables extensibility** — Unknown fields should be ignored, not rejected
3. **Has a dark side** — Masks bugs, enables specification rot, creates security risks
4. **Context matters** — Liberal for interop and extensibility; strict for security
5. **Modern balance** — Validate required fields strictly, ignore unknowns, output predictably

---

Resilience & Graceful Degradation

↑ top

> "In complex systems, failure is the normal state. Success is the special case that requires explanation." — Richard Cook

Core Concept

Continue operating despite partial failures. Anticipate failure, implement recovery, degrade gracefully.

The Three Pillars

Pillar	Purpose	Mechanism
Retry	Recover from transient failures	Exponential backoff with jitter
Fallback	Provide degraded service	Cached data, default values
Protect	Prevent cascade failures	Circuit breakers, timeouts

Pattern 1: Exponential Backoff with Jitter

```
delay = min(base_delay * 2^attempt + random_jitter, max_delay)
```

Pattern 2: Circuit Breaker

Three states: CLOSED (normal) → OPEN (fail fast) → HALF-OPEN (test recovery)

Pattern 3: Graceful Degradation

```python
# Cascading fallback strategy
def get_recommendations(user_id: str) -> list[Product]:
    try:
        return recommendation_service.get_personalized(user_id)
    except ServiceUnavailableError:
        cached = cache.get(f"recommendations:{user_id}")
        if cached:
            return cached
        return get_popular_items()  # Final fallback
```

Summary

1. **Failures are inevitable** — Design for them, don't assume success
2. **Retry with exponential backoff and jitter** — Prevents thundering herd
3. **Only retry transient errors** — Auth failures should fail fast
4. **Use circuit breakers** — Prevent cascading failures
5. **Always set timeouts** — Unbounded waits exhaust resources

---

Principle of Least Privilege

↑ top

> "Every program and every user of the system should operate using the least set of privileges necessary to complete the job." — Jerome Saltzer, *Protection and the Control of Information Sharing in Multics* (1974)

Core Concept

Minimum permissions necessary for intended function—nothing more.

1. **Minimize Attack Surface** — Fewer permissions = fewer entry points
2. **Limit Blast Radius** — Contain damage when breaches occur

74% of breaches start with privileged credential abuse.

Application at Every Level

Level	Example
Function	A function that reads config shouldn't have write access
Class	A `ReportGenerator` shouldn't have user deletion capabilities
Service	A payment microservice shouldn't access user profile data
Account	A database user for reads shouldn't have DROP TABLE privileges

Real-World Failures

Breach	What Happened	PoLP Failure
Equifax (2017)	143M records stolen; attackers executed 9,000 DB queries	Permissive access controls; no network segmentation
Target (2013)	40M credit cards via HVAC vendor	Third-party had excessive network access

Common Violations

Code Smells: Service accounts with `*` wildcard permissions, database connections with admin privileges, shared credentials across services, functions that accept more capabilities than needed.

Verbal Cues: "Just give it admin access, it's easier", "We'll lock it down later", "It needs these permissions for debugging"

Anti-Patterns

```python
#  Wrong - Over-privileged database connection
def get_user_email(user_id: int) -> str:
    conn = get_admin_connection()  # Has DELETE, DROP, etc.
    return conn.execute("SELECT email FROM users WHERE id = ?", user_id)

#  Correct - Minimal privileges for the task
def get_user_email(user_id: int) -> str:
    conn = get_readonly_connection()  # Only SELECT privilege
    return conn.execute("SELECT email FROM users WHERE id = ?", user_id)
```

```python
#  Wrong - Function accepts overly broad context
def send_notification(user: User, db: DatabaseAdmin):
    email = db.query(f"SELECT email FROM users WHERE id = {user.id}")
    # db could delete the entire users table!

#  Correct - Function receives only what it needs
def send_notification(email: str):
    send_email(email, "Your notification...")  # Cannot access database
```

```yaml
#  Wrong - IAM policy with wildcard
Effect: Allow
Action: "s3:*"
Resource: "*"

#  Correct - Scoped to specific actions and resources
Effect: Allow
Action: ["s3:GetObject", "s3:PutObject"]
Resource: "arn:aws:s3:::my-bucket/uploads/*"
```

Implementation Strategies

Strategy	Description
Default deny	Start with no access, explicitly grant what's needed
Separate accounts by function	Different credentials for read vs. write operations
Time-bounded access	Temporary elevated privileges that expire
Audit unused permissions	Regularly review and remove permissions not being used

Privilege Creep

Permissions accumulate beyond current needs: role changes without revocation, temporary access becoming permanent, misleading role names.

Prevention: Regular access reviews, automated permission expiration, minimal scope at design time.

Relationship to Zero Trust

Framework	Focus
Zero Trust	Verify identity ("never trust, always verify")
Least Privilege	Limit access ("need to know")

Partners: Zero Trust authenticates requests; PoLP limits authenticated access. Defense in depth.

Summary

1. **Grant minimum necessary permissions** — Start with nothing, add only what's required
2. **Scope permissions tightly** — Specific resources, specific actions, specific time windows
3. **Separate credentials by function** — Read-only users for reads, write users for writes
4. **Audit and prune regularly** — Permissions accumulate; actively remove unused access
5. **Design for minimal access** — Functions, classes, and services should request only what they need

---

Maintainability & Operations

↑ top

Code is a living artifact. The Boy Scout Rule keeps code improving incrementally with every change, while Observability & Transparency ensure you can understand what your systems are doing in production.

Boy Scout Rule

↑ top

> "Always leave the code better than you found it." — Robert C. Martin (Uncle Bob), *Clean Code*

Core Concept

With each commit, leave code slightly better than you found it. Without active maintenance, technical debt accumulates. Continuous small improvements beat periodic "refactoring sprints."

Why It Works

Traditional Approach	Boy Scout Rule
Accumulate debt, then "refactoring sprint"	Continuous small improvements
Cleanup disrupts feature delivery	Cleanup happens alongside features
Requires dedicated time allocation	Built into every commit
Big changes = big risk	Small changes = low risk
Code rot between sprints	Code improves continuously

What "Better" Looks Like

Small improvements that take seconds to minutes:

Category	Examples
Naming	Rename `$a` to `$account`, `proc()` to `process_order()`
Cleanup	Remove unused imports, dead code, extra blank lines
Deprecations	Replace deprecated API calls with current alternatives
Formatting	Fix inconsistent indentation, add missing whitespace
Duplication	Extract repeated logic into a helper (if pattern is proven)
Clarity	Simplify a complex conditional into a named method

The Campground, Not the Forest

Clean the campground, not the entire forest.

```
#  Wrong - Changed 350 files to remove blank lines project-wide
# This makes code review impossible and introduces huge risk

#  Correct - Cleaned up the file you're actually working in
def process_order(order_id: int) -> Order:
    # While adding this method, noticed and fixed:
```

```
    # - Renamed 'o' to 'order'
    # - Removed unused import
    # - Fixed inconsistent indentation
    order = self.repository.get(order_id)
    return self.apply_discount(order)
```

Scope cleanup to files you're already touching. Issues elsewhere? Create a ticket.

Common Violations

Code Smells Left Behind: - Ignoring deprecation warnings - Leaving unused variables/imports - Not fixing obvious naming issues - Copying code instead of extracting

Verbal Cues: - "I'll clean it up later" (you won't) - "That's not my code" - "It works, don't touch it" - "We need a refactoring sprint"

When NOT to Apply

Exceptions: - **Unfamiliar code**: Don't "improve" code you don't fully understand - **No test coverage**: Risky refactors in untested code can introduce bugs - **Time-critical fixes**: Production incidents need the fix, not cleanup - **Shared/external code**: Extra care when changes affect other teams

Clean up obvious issues; for larger concerns, create a ticket.

Anti-Patterns

```
#  Wrong - "Not my problem" attitude
def add_discount(order):
    # Just adding my feature, ignoring the mess
    o = order  # terrible variable name from legacy code
    d = o.total * 0.1  # magic number
    o.total = o.total - d
    return o

#  Correct - Boy Scout approach
def add_discount(order: Order) -> Order:
    # Cleaned up while adding feature:
    # - Renamed variables for clarity
    # - Extracted magic number to constant
    DISCOUNT_RATE = 0.1
    discount = order.total * DISCOUNT_RATE
    order.total = order.total - discount
    return order
```

Relationship to Other Principles

Principle	Connection
Broken Windows Theory	Boy Scout Rule is the *antidote*—fix small issues before they invite bigger ones
DRY	Boy Scout Rule helps you spot and fix duplication incrementally
Self-Documenting Code	Rename unclear variables as you encounter them
YAGNI	Delete unused code when you find it
Opportunistic Refactoring	Same concept, different name—improve code while you're there
Technical Debt	Boy Scout Rule is continuous debt payment

Broken Windows: Neglect invites more neglect. Boy Scout Rule signals "this code is cared for."

Summary

1. **Leave code better than you found it** — Every commit is an opportunity
2. **Small improvements compound** — Minutes daily beats weeks annually
3. **Clean the campground, not the forest** — Scope to files you're touching
4. **Don't ignore the mess** — "Not my code" is not an excuse
5. **Make cleanup socially expected** — It should be as unacceptable to leave mess as to litter

---

Observability & Transparency

↑ top

> "Observability is the ability to understand the internal state of a system by examining its external outputs." — Charity Majors

Core Concept

Make system behavior visible through structured telemetry. In distributed systems, observability is your primary debugging tool.

The Three Pillars

1. **Logs**: Chronological records of discrete events with context

2. **Metrics**: Quantitative measurements over time
3. **Traces**: End-to-end journey of requests through distributed systems

Observability Principles

1. **Structured Over Unstructured**: Use JSON, key-value pairs
2. **Semantic Prefixes**: Emojis for quick visual scanning
3. **Log Levels Match Intent**: DEBUG, INFO, WARNING, ERROR, CRITICAL
4. **Context Flows Through Systems**: Trace IDs, request IDs
5. **Metadata in Responses**: Return operational metadata
6. **Timing Everything Important**: Instrument performance-critical paths

Anti-Patterns

- **Silent Failures**: Swallowed exceptions
- **Opaque Error Messages**: Generic, unhelpful messages
- **Missing Request Context**: No correlation IDs
- **Over-Logging**: Logging inside tight loops
- **Logging Sensitive Data**: Credentials in logs

Summary

1. **Observability is your debugger** in production
2. **Structure your logs** for machine parsing and human readability
3. **Propagate context** (request IDs, trace IDs)
4. **Return metadata** in responses for transparency
5. **Never log secrets** — sanitize sensitive data

When to Relax Rules

↑ top

Over-applying principles causes as much harm as ignoring them. Know when to make exceptions.

Context	Relaxed Principles	Why
Prototypes/Spikes	All	Exploring, not building. Throw it away.

Context	Relaxed Principles	Why
Test Code	DRY	DAMP (Descriptive And Meaningful Phrases) > DRY. Readability trumps deduplication.
Performance-Critical	Abstractions, DI	Hot paths may need inlining. Profile first.
Scripts < 100 lines	Modularity, SRP	Overhead exceeds benefit. Keep it simple.
Glue Code	Most patterns	Thin integration layers don't need architecture.
Generated Code	All	Don't hand-edit generated code. Fix the generator.
Legacy Migration	Boy Scout	Large refactors need dedicated effort, not incremental changes.

Context	Relaxed Principles	Why
Data Transfer Objects	Encapsulation	DTOs are meant to expose data. That's their job.
Configuration	YAGNI	Config flexibility is often worth it—cheaper than redeployment.
Security Boundaries	Postel's Law	Be paranoid, not liberal. Validate everything strictly.

The Meta-Principle

> **"Rules are for the guidance of wise men and the obedience of fools."** — Douglas Bader

Principles are heuristics, not laws. Understand WHY before applying. If following makes code worse, don't.

References

↑ top

Foundational Texts

Essential books that shaped modern software design thinking.

Book	Author(s)	Key Contribution
The Pragmatic Programmer	Andy Hunt & Dave Thomas	Practical heuristics including DRY, orthogonality, tracer bullets, and "Tell Don't Ask"
Clean Code	Robert C. Martin	Function size, naming, and the Single Responsibility Principle

Book	Author(s)	Key Contribution
A Philosophy of Software Design	John Ousterhout	Deep vs. shallow modules, complexity as the root problem, strategic vs. tactical programming
Design Patterns	Gang of Four (Gamma, Helm, Johnson, Vlissides)	23 reusable OO patterns; established patterns vocabulary
Object-Oriented Software Construction	Bertrand Meyer	Design by Contract, Command-Query Separation, Open-Closed Principle
Refactoring	Martin Fowler	Systematic code improvement techniques; code smells catalog
Working Effectively with Legacy Code	Michael Feathers	Seams, characterization tests, safely changing untested code
Domain-Driven Design	Eric Evans	Ubiquitous language, bounded contexts, strategic design

Seminal Articles & Essays

Influential writings that introduced or crystallized important concepts.

Article	Author	Year	Key Idea
Parse, Don't Validate	Alexis King	2019	Transform unstructured data into types that prove validity; let the type system enforce invariants
The Wrong Abstraction	Sandi Metz	2016	"Duplication is far cheaper than the wrong abstraction"; prefer inline code over premature DRY

Article	Author	Year	Key Idea
Cognitive Load is What Matters	Artem Zakirullin	2023	Minimize mental effort required to understand code; endorsed by Rob Pike, Andrej Karpathy
Tell Don't Ask	Martin Fowler	2013	Tell objects what to do rather than asking for data and acting on it
Four Rules of Simple Design	Kent Beck	~1990s	(1) Passes tests, (2) Reveals intention, (3) No duplication, (4) Fewest elements
Law of Demeter	Karl Lieberherr et al.	1987	Only talk to your immediate friends; minimize coupling chains

Online Resources

Living references for patterns, principles, and refactoring techniques.

- Martin Fowler's Bliki — Authoritative essays on patterns, refactoring, and architecture
- Refactoring Guru — Visual catalog of design patterns and refactoring techniques
- DevIQ Principles — Concise summaries of software development principles
- c2 Wiki (Cunningham & Cunningham) — The original patterns wiki; historical discussions on OO design
- Source Making — Design patterns, anti-patterns, and refactoring catalog

Concept Attribution

Origins of specific principles referenced in this document.

Concept	Origin
KISS	U.S. Navy, 1960s; popularized by Kelly Johnson (Lockheed Skunk Works)
YAGNI	Extreme Programming (Kent Beck, Ron Jeffries), late 1990s
DRY	*The Pragmatic Programmer* (Hunt & Thomas), 1999

Concept	Origin
SOLID	Robert C. Martin, early 2000s (acronym coined by Michael Feathers)
Separation of Concerns	Edsger Dijkstra, 1974
Design by Contract	Bertrand Meyer, 1986 (Eiffel language)
Postel's Law	Jon Postel, RFC 761 (TCP), 1980
Deep Modules	John Ousterhout, *A Philosophy of Software Design*, 2018
Rule of Three	Folk wisdom; formalized in *Refactoring* (Fowler)
Cognitive Load	Psychology (John Sweller, 1988); applied to code by Zakirullin, 2023

This document is designed to be dropped into your `~/.claude/commands/` and/or `~/.opencode/commands` folders. Run `/deslop [file-or-directory]` to analyze your code against these principles and let the agent fix any slop it detects.